

RESEARCH ARTICLE

Students' Experiences of Using ChatGPT in an Undergraduate Programming Course

PHILIPP HAINDL^{ID} AND GERALD WEINBERGER^{ID}

Department of Computer Science and Security, St. Pölten University of Applied Sciences, 3100 St. Pölten, Austria

Corresponding author: Philipp Haindl (philipp.haindl@fhstp.ac.at)

ABSTRACT Increasing use of artificial intelligence tools in programming education calls for a deeper understanding of their effect on students' learning. This paper presents a study that investigates the experiences of part-time undergraduate students using ChatGPT in a five-week Java programming course. After each exercise, students provided feedback via anonymous surveys in which they rated different suitability aspects of ChatGPT. The majority viewed ChatGPT positively and suitable for learning programming concepts. However, its suitability for specific implementation tasks received mixed reviews. Students found it easy to adapt ChatGPT's generated code to the exercises' implementation tasks. The students primarily used it for acquiring background knowledge, learning syntax and programming concepts and suggesting suitable algorithms. Yet, some abstained from using it due to concerns to not garner sufficient programming proficiency, retrieving partially incorrect or misleading generated code, preferring an independent working style, or general skepticism about its benefits. Finally, in response to our findings, we also discuss three perspective directions for improving the suitability of LLM chatbots for students in programming education.

INDEX TERMS Programming education, ChatGPT, generative AI, large language models.

I. INTRODUCTION

Chatbots are computer programs that mimic human-like conversation with users using natural language or text, thus creating a sense of interpersonal interaction [1], [2]. Earlier versions of chatbots relied on rudimentary pattern matching and textual analysis, but modern versions use knowledge-based models [3]. Chatbots have been integrated into formal and informal education for a number of years [4], [5]. As part of their applications, they support the learning process [6], [7], enhance student engagement [8], [9], evaluate students' progress [10], [11], and perform administrative tasks [12], [13]. However, educational chatbots face a range of constraints and obstacles [14], [15].

ChatGPT makes use of *Large Language Models* (LLMs) to facilitate conversation. By using LLMs, it is equipped with a vast linguistic knowledge base that enables it

to comprehend and generate human-like text. In the era of ChatGPT, programming education must determine the best use cases for integrating and applying chatbots in the classroom. As we embrace new programming tools, it is imperative that students are capable of evaluating the quality of AI-generated code and text responses while also developing programming fluency and critical thinking skills. As an example of the needed shift in programming education, Ozkaya emphasized the following: *"We teach students hello world development, while we should be teaching them how to read millions of lines of code, triage and fix bugs ... we need to be teaching next-generation software engineers when to trust, how to create evidence to trust, how to do trust assessment rapidly and correctly, and how to improve such assistants"* [16]. The need for engineers to critique and refine their work is emphasized by future software development practices like *"reflective, intelligent software development"* or *"design prompt-driven iterative development"* [17]. Math education's history with technology illustrates the importance of maintaining fundamental skills regardless of evolving

The associate editor coordinating the review of this manuscript and approving it for publication was Utku Kose^{ID}.

tools [18]. In his opinion paper, Welsh [19] even imagines a future where traditional programming will become obsolete because future programmers will train AI models instead of focusing on classic programming tasks. The shift is comparable to how today's programmers lack knowledge of low-level hardware details.

To investigate students' experience when using ChatGPT in an introductory course in Java programming, we formulated the following research questions:

- **(RQ1)** How do students rate the effect of ChatGPT on their learning progress between programming exercises?
- **(RQ2)** How do students rate ChatGPT's suitability for implementation tasks **(RQ2.1)**, for learning programming concepts **(RQ2.2)**, and its overall suitability in a programming exercise **(RQ2.3)**?
- **(RQ3)** What additional effort is required by students to adapt ChatGPT's generated code to the concrete implementation tasks in a programming exercise?
- **(RQ4)** What are common application scenarios students use ChatGPT for in a programming exercise?
- **(RQ5)** What are common reasons of students for not using ChatGPT in a programming exercise?

To this end, the research presented in this paper contributes

- an evaluation of ChatGPT's
 - effect on students' learning progress
 - suitability for implementation tasks
 - suitability for learning programming concepts
 - overall suitability
 - an evaluation of the required effort to adapt ChatGPT's generated code to implementation tasks
 - common application scenarios of ChatGPT
 - common reasons for not using ChatGPT
- in a programming exercise from students' perspective.

The paper is structured as follows: We elaborate works related to our study in Section II and describe the used research methods in Section III. Subsequently, in Section IV, we present the results of our study and discuss perspective directions for improving the suitability of LLM chatbots in programming education in Section V. Afterwards, we describe limitations of our study in Section VI and conclude our work by also highlighting future research directions in Section VII.

II. RELATED WORK

An emerging body of research explores the applicability of ChatGPT and similar LLMs in higher education programming courses.

Rahman and Watanobe [20] assessed the implications of ChatGPT for programming education. Using an online rating system, the authors evaluated the accuracy of answers provided by ChatGPT related to code generation, pseudocode creation, and code correction experiments. A subsequent survey was conducted with students and lecturers to assess the educational support provided by ChatGPT. Although ChatGPT and similar chatbots have enormous potential

for education and research, the authors also stressed their notable limitations. These include a lack of common sense, potential biases, and challenges in complex reasoning. Based on a controlled experiment, Kazemitabaar et al. [21] studied the supportiveness of AI code generators for novice programming learners between 10 and 17 years old. During the experiment, half of the learners had access to AI code generators and performed Python code-authoring tasks. After a week, learners who used AI code generators performed better in code-authoring performance, completion rate, and test evaluation than those who didn't. Due to sample size, some results did not reach statistical significance, and the findings might not apply to learners of other ages. The authors call for further research on how AI code generators support algorithmic thinking in software education and learner behavior. Based on a literature review, Opara et al. [22] assessed ChatGPT's capabilities and utility in education and research. Based on the authors' findings, ChatGPT supports active and self-paced learners who tightly integrate ChatGPT into their learning routines. According to the authors, it hinders the learning progress because it inhibits the learners' creative mindset and provides inaccurate responses. A study conducted by Bull and Kharrufa [23] explored current practices, challenges, and opinions regarding generative AI tools in industrial software development through exploratory interviews with software professionals. The interviews were analyzed qualitatively to formulate visions for software engineering education. According to the interviewees, their organizations predominantly use tools such as ChatGPT and GitHub Copilot. As drawbacks of these tools, participants consistently expressed concerns about outdated training data and a lack of understanding of the larger project context. Therefore, these tools often provide confident, but incorrect answers or answers that are outdated with the latest APIs. Daun and Brings discussed ChatGPT's benefits and challenges in their position paper [24]. Using ChatGPT, they demonstrated a number of activities that are typically used in programming courses, such as generating code, comparing technologies, and searching literature. In student-ChatGPT interactions, there were several examples of misleading answers to specific questions. The authors emphasize that students should be guided and supervised to evaluate answers when using ChatGPT. Personalized feedback was also emphasized as an important advancement for software engineering education by the authors. To determine the effectiveness of ChatGPT for undergraduate Java programming courses, Ooh et al. [25] analyzed 80 ChatGPT generated solutions. When presented with non-textual descriptions, class files, API documentation, and UML diagrams, ChatGPT was found to have limitations. When no complex instructions or non-textual elements were present in student exercises, ChatGPT generated code fragments that are readable and supportive. Savelka et al. [26] examined the ability of generative pre-trained transformers, specifically `text-davinci-003`, to pass programming

assessments in introductory and intermediate Python courses. In a similar manner to Ouh et al. [25], their study revealed the limitations of LLM models when the exercises require complex chains of reasoning, non-trivial reasoning about code snippets, or if code snippets need to be adjusted after comparing actual and expected results. Despite not covering all aspects of programming assessment, current models enabled students to achieve a non-negligible portion of the course's overall score. As a result, the authors suggest integrating requirements formulation, debugging, trade-off analysis, and critical thinking in programming assessments.

It can be summarized that only two studies [20], [21] incorporated students' experiences about using ChatGPT in programming education. The findings presented in our paper contribute to this lack of empirical evidence of ChatGPT's suitability from students' perspective for an introductory programming course.

III. METHOD

We conducted our study in a Java programming course with 22 part-time bachelor students in information security. This programming course lasted five weeks, following the previous semester's Python programming course. None of the students had previous knowledge of the course's contents.

A. COURSE STRUCTURE

Students were required to attend five on-campus lectures, complete the exercises at home, and take a written test at the end of the course. The exercises covered basic concepts of the Java programming language, i.e., "*Object-Oriented Programming*", "*Interfaces and Exception Handling*", "*Collections*", "*File IO and Streams*", and "*Lambda Expressions and Multithreading*". After one week, students submitted their implementations using our university's online teaching system. The course's lecturer reviewed and graded the students' exercises and provided individual feedback.

Each exercise was worth 50 points. Although exercise submission was not required, students were highly encouraged to prepare and submit the exercises for feedback from the lecturer.

B. STUDENT BRIEFING

The concrete objectives and structure of the study were explained to students at the beginning of the programming course. Furthermore, they were informed about the anonymity of their data and how it would be analyzed. As part of this briefing, we also explained the questionnaire structure in detail to ensure that each question was understood uniformly and correctly. It was explained to them that using ChatGPT when preparing the exercises is optional, and they can decide whether to use it for each exercise. Upon briefing the students about our study, we sought their consent to participate. It was our intention to ensure that any concerns arising during the study could be anonymously reported to us. In this regard, we asked the class representative to repeatedly

ask his colleagues and inform us if there are any questions. During the study, however, no concerns or requests were raised by students.

C. QUESTIONNAIRE

After submitting their exercises' code, students were required to complete a 12-question anonymous online survey (see Table 1). It included three open text questions, one yes/no question, one single choice question and two multiple choice questions, as well as five questions with 5-point Likert scale answers. Students who answered that they did not use ChatGPT for the respective programming exercise (Q2), they were asked to give the reason for not using it (Q3) and all other questions were skipped. We explained to students that when rating the effect of ChatGPT on their learning progress (Q6) this rating should be attributed exclusively to ChatGPT, not in general. If students answered that ChatGPT was used for another application scenario than those listed (Q11), we asked them to describe it (Q12). The provided application scenarios for ChatGPT comprised the acquisition of background knowledge, the learning of Java syntax and programming concepts, suggestion of suitable algorithms (based on an existing solution approach), solution ideation (non-iterative), creation of the first solution approach (iterative), improving and review of the solution approach (iterative). An *iterative* application scenario is characterized by repeated interactions between the student and ChatGPT in order to reach a final solution iteratively.

D. ANALYSIS

Following the course, the closed questions (Q1, Q2, Q4-Q11) of the survey were quantitatively analyzed. For each of the five exercises, the two open text answers describing the reasons for not using ChatGPT (Q3) and missing application scenarios (Q12) were analyzed qualitatively, in case students responded to these questions. Using a grounded theory approach, we read all statements of the students and codified recurring themes. Grounded theory involves inferring theory from empirical data, such as interviews or surveys [27]. It was chosen for its strength in identifying and conceptualizing underlying patterns and themes within qualitative data, particularly suitable for exploring new or under-researched areas. During the *initial coding* phase, we utilized a subset of student responses to develop a preliminary code set. This stage was critical for identifying preliminary themes and ensuring that our analysis would accurately and comprehensively capture the students' perspectives. We thoroughly discussed any discrepancies to enhance the reliability of our code set. Subsequently, in the *focused coding* phase, we tested the preliminary code set against a larger subset of students' open text answers. This process enabled us to refine our codes further and ensure they remained grounded in the empirical data. We consistently conducted periodic reviews to maintain coding consistency and resolve any emerging discrepancies, thereby ensuring the rigor of the coding process.

TABLE 1. Questionnaire structure.

ID	Question	Answer option
Q1	How many hours have you spent preparing this exercise?	Open text
Q2	Have you used ChatGPT for this exercise (even partially)?	Yes/No
Q3	What is the reason you haven't used ChatGPT for this exercise?	Open text
Q4	For which implementation tasks of this exercises have you used ChatGPT?	Multiple-Choice
Q5	Which version of ChatGPT have you used?	Single-Choice (v3.5/v4)
Q6	How do you rate ChatGPT's effect on your learning progress compared to the last exercise?	5-point Likert (diverging)
Q7	How do you rate the suitability of ChatGPT for implementation tasks of this exercise?	5-point Likert (diverging)
Q8	How do you rate the suitability of ChatGPT for learning programming concepts of this exercise?	5-point Likert (diverging)
Q9	How do you rate the overall suitability of ChatGPT for this exercise?	5-point Likert (diverging)
Q10	How much additional effort was required to adapt ChatGPT's generated code to the implementation tasks of this exercise?	5-point Likert (increasing)
Q11	For which of the given application scenarios have you used ChatGPT in this exercise?	Multiple-Choice
Q12	For which other application scenarios have you used ChatGPT in this exercise?	Open text

In the five programming exercises, students were asked to provide Likert ratings, which resulted in ordinal data being analyzed (Q6-Q10). The datasets are paired since the same group of students was repeatedly asked to give their ratings for the different exercises. Friedman's test is a non-parametric statistical test designed to detect variations among groups of related subjects, making it particularly appropriate for small sample sizes, ordinal data, and repeated measures on the same group of participants. Using the Friedman test we can determine whether the Likert ratings of the examined ChatGPT's qualities are statistically significant across the five programming exercises.

The statistical significance was set to $p < 0.05$ and we formulated the following null hypotheses $H_{RQ1-RQ3,0}$ for the respective research questions: *There is no correlation between the programming exercises and students' ratings of ChatGPT's*

- $H_{1,0}$: effect on their learning progress.
- $H_{2,1,0}$: suitability for implementation tasks.
- $H_{2,2,0}$: suitability for learning programming concepts.
- $H_{2,3,0}$: overall suitability.

and $H_{3,0}$ as: *There is no correlation between the programming exercises and the additional efforts required to fit ChatGPT's generated code to the exercises' tasks.* As a result, the corresponding alternative hypotheses $H_{RQ1-RQ3,a}$ can be accepted with a p -value of less than 0.05, indicating that there is a statistically significant correlation between the Likert ratings of the examined ChatGPT's qualities across the five programming exercises.

IV. RESULT

Students could choose whether to use ChatGPT or not for each programming exercise. Those who have not used it were asked in the survey to explain why. Table 2 shows the students' submissions for each programming exercise, their use of ChatGPT or not use of it, and the time they spent preparing for it.

TABLE 2. Students' exercise submissions.

Exercise focus	ChatGPT usage		preparation time	
	used	not used	mean	sd
Object-Oriented Programming	19	3	12.78	5.43
Interfaces, Exception Handling	19	3	9.39	4.84
Collections	15	6	8.57	4.21
File, IO Streams	16	3	8	3.97
Lambda Expressions, Multithreading	12	6	7.06	3.66

For a positive grade, not all five exercises had to be submitted, but the threshold was set at 60% of the total points across all exercises. The "Collections" exercise has not been submitted by one student, and the "File IO and Streams" exercise has not been submitted by three students. The "Lambda Expressions and Multithreading" exercise was not submitted by four students.

In total, the students used GPT 3.5 for 54 exercises (66,6%), and GPT 4.0 for 27 exercises (33,3%). Following, we present the results of our study in the sequence of the research questions.

A. EFFECT OF CHATGPT ON STUDENTS' LEARNING PROGRESS (RQ1)

For answering question Q6 in the survey, students were explicitly asked to consider only that portion of the learning effect that they attribute to ChatGPT. As depicted in Figure 1, the overall majority of students rated ChatGPT as either *rather positive* or *positive* in regards to its effect on their learning progress. There was no negative rating from any student. Figure 1 also shows that the first two programming exercises that dealt with "Object-Oriented Programming" and "Interfaces and Exception Handling" were rated more positively overall than the others. However, it should be noted that these two exercises received the most

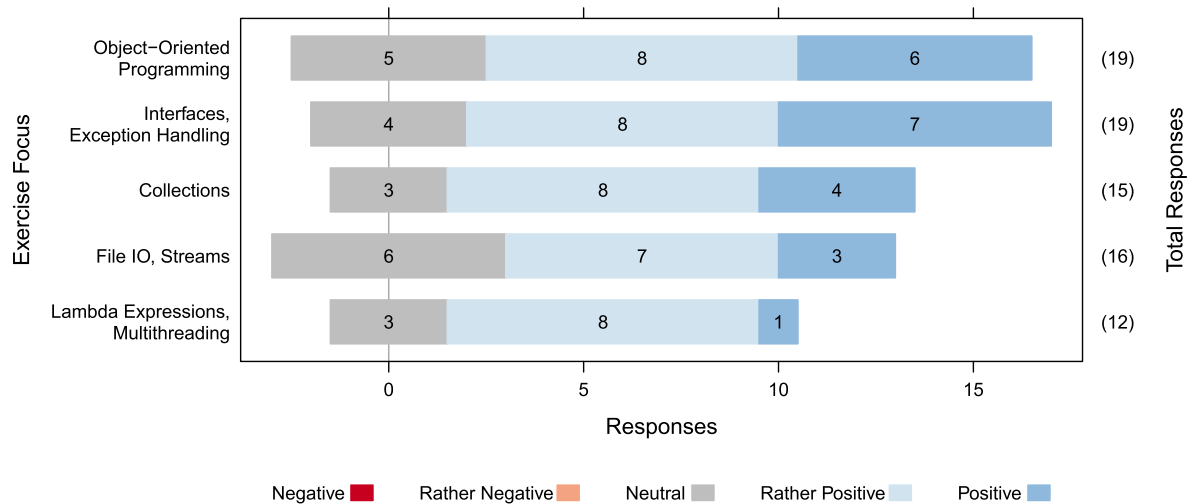


FIGURE 1. Students' ratings of ChatGPT's effect on their learning progress.

ratings overall. For the “File IO and Streams” programming exercise there was a higher proportion of students rating ChatGPT's effect on their learning progress as *neutral* than on any other exercise.

Based on the Friedman test, the *p-value* of the students' ratings across the programming exercises is 0.2311. Accordingly, the null hypothesis $H_{1,0}$ can be accepted stating that there is no relationship between the programming exercises and the students' ratings of ChatGPT's effect on their learning progress.

B. SUITABILITY OF CHATGPT FOR DIFFERENT LEARNING ACTIVITIES (RQ2)

On a 5-point Likert scale, the students were asked to rate the suitability of ChatGPT for implementation tasks and learning programming concepts.

1) IMPLEMENTATION TASKS

Figure 2 shows that, except for the exercise dealing with object-orientation, the majority of students considered ChatGPT to be *suitable* or *rather suitable* for implementing programming exercises. In this regard, “suitable for implementation tasks” means that, from students' perspective, they could use ChatGPT's generated code for the implementation tasks in a meaningful manner, e.g., without considerable adaptations. There were negative ratings of ChatGPT's suitability for this aspect in all exercises, i.e., as either *not suitable* or *rather not suitable*. This indicates that students aren't fully convinced about ChatGPT's suitability for implementation tasks. For the exercises “Interfaces and Exception Handling” and “File IO and Streams”, students rated ChatGPT's suitability for implementation tasks best, while for exercise “Object-Oriented Programming” they rated it worst with one student (5%) as even *not suitable*.

Friedman's test yields a *p-value* of 0.4928 for the students' ratings of this suitability aspect. Accordingly,

there is no statistically significant relationship between the programming exercises and the students' ratings of ChatGPT's suitability for implementation tasks. Hence, the null hypothesis $H_{2,1,0}$ can be accepted.

2) LEARNING PROGRAMMING CONCEPTS

Figure 3 shows a different picture when students were asked whether ChatGPT supports their theoretical understanding of programming concepts. Students generally rated ChatGPT as *rather suitable* or *suitable* to support them in learning programming concepts across all exercises. For the “Object-Oriented Programming” and “File IO and Streams” exercises only, three students in total (one for “Object-Oriented Programming” and two for “File IO and Streams”) rated ChatGPT as *rather not suitable*. We believe this indicates that students are more confident that ChatGPT can help them learning programming concepts than for the exercises' implementation tasks.

Friedman's test gives a *p-value* of $1e-04$. As a result, the alternative hypothesis $H_{2,2,a}$ can be accepted. This hypothesis states that there is a significant relationship between programming exercises and students' ratings of ChatGPT's suitability for learning programming concepts.

3) OVERALL SUITABILITY

Figure 4 depicts the students' ratings of ChatGPT's overall suitability in the programming exercises. The results of this research question are similar to those of the previous research question. Students rated ChatGPT's overall suitability for the programming exercises as predominantly positive and higher than for implementation tasks and learning programming concepts individually. Only one student (5%) rated ChatGPT's overall suitability for the “Object-Oriented Programming” and “Interfaces and Exception Handling” exercises as *rather not suitable*.

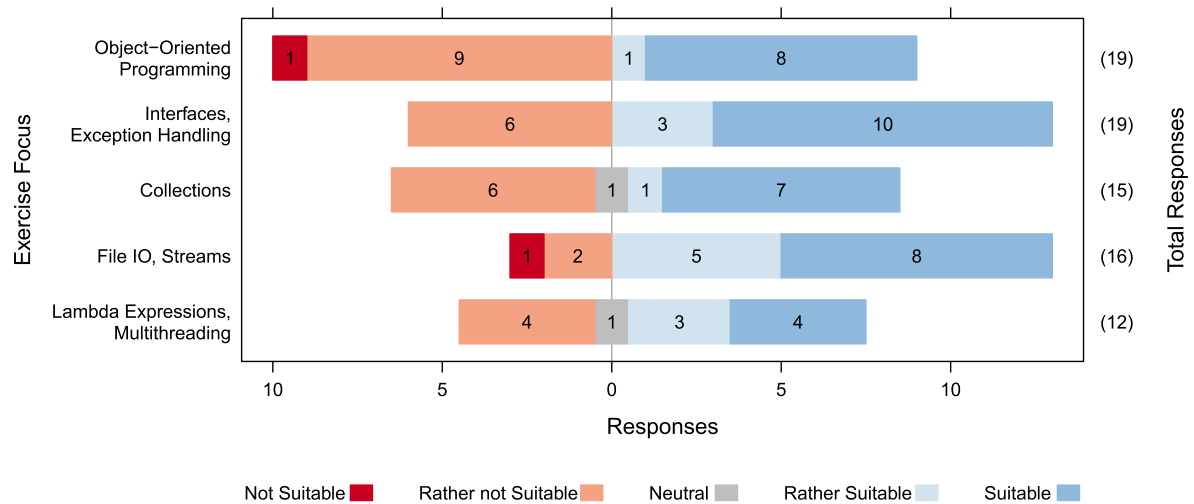


FIGURE 2. Students' ratings of ChatGPT's suitability for implementation tasks.

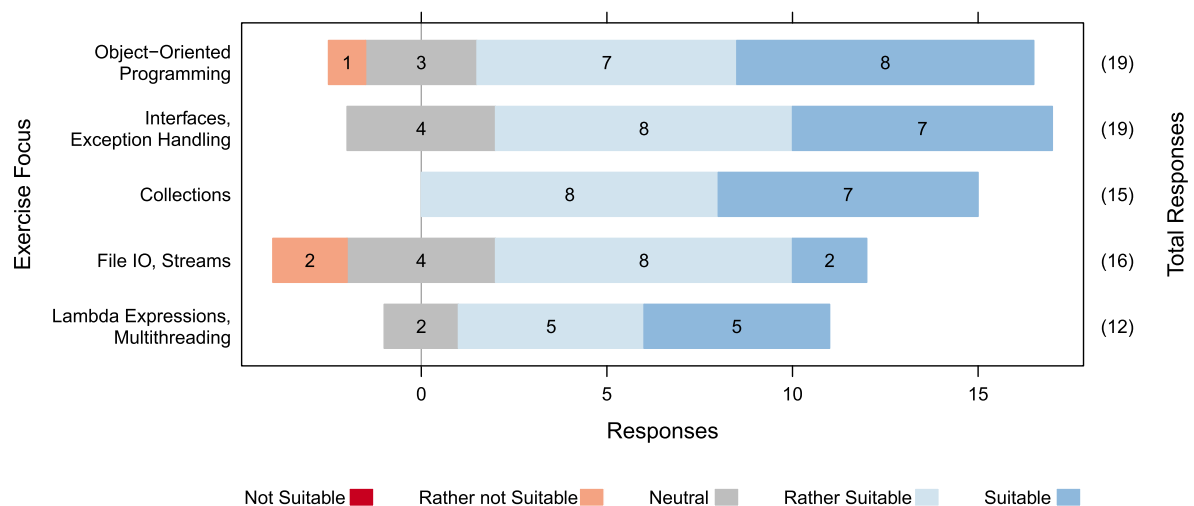


FIGURE 3. Students' ratings of ChatGPT's suitability for learning programming concepts.

Friedman's test gives a p -value of 0.0002856 based on the students' ratings, thus there is a significant correlation between the programming exercises and students' ratings of the overall suitability of ChatGPT in the exercises. Hence, we can accept the alternative hypothesis $H_{2.3,a}$ in this case.

C. ADDITIONAL REQUIRED EFFORT TO ADAPT CHATGPT'S GENERATED CODE (RQ3)

On a 5-point Likert scale with items increasing from *none* to *very high* students were asked to rate the additional effort required to adapt ChatGPT's generated code to the implementation tasks in the exercises (cf. Figure 5). For all exercises, only a few students had to spend either no extra effort or *high* respectively *very high* additional effort to adapt ChatGPT's generated code. For the most part, ChatGPT's generated code could be adapted to the implementation tasks with only *very little* or *little* extra effort. The additional

efforts decreased after the fourth programming exercise ("*Collections*"), which might also be attributed to students becoming more familiar with ChatGPT and formulating more appropriate prompts.

Friedman's test gives a p -value of 0.3666 based on students' ratings. Hence, the null hypothesis $H_{3,0}$ can be accepted, declaring that there is no correlation between the programming exercises and the additional efforts required to adapt ChatGPT's generated code to the exercises' implementation tasks.

D. APPLICATION SCENARIOS OF CHATGPT (RQ4)

Students were asked to select the most common application scenarios for which they have used ChatGPT to prepare the programming exercises. Multiple application scenarios could be selected at the same time. Students had six closed-answer options and one open-text option in case the mentioned

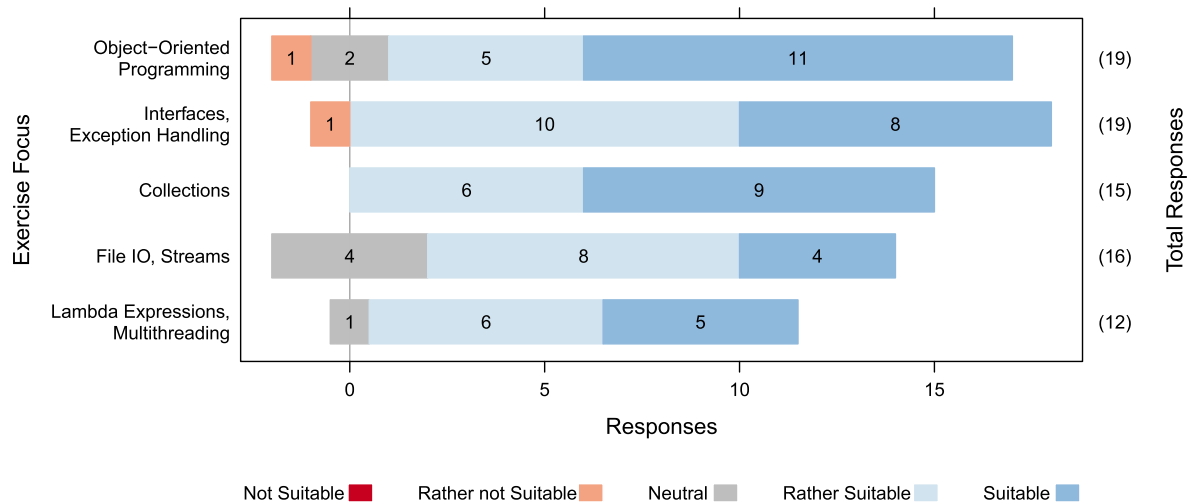


FIGURE 4. Students' ratings of ChatGPT's overall suitability for the programming exercises.

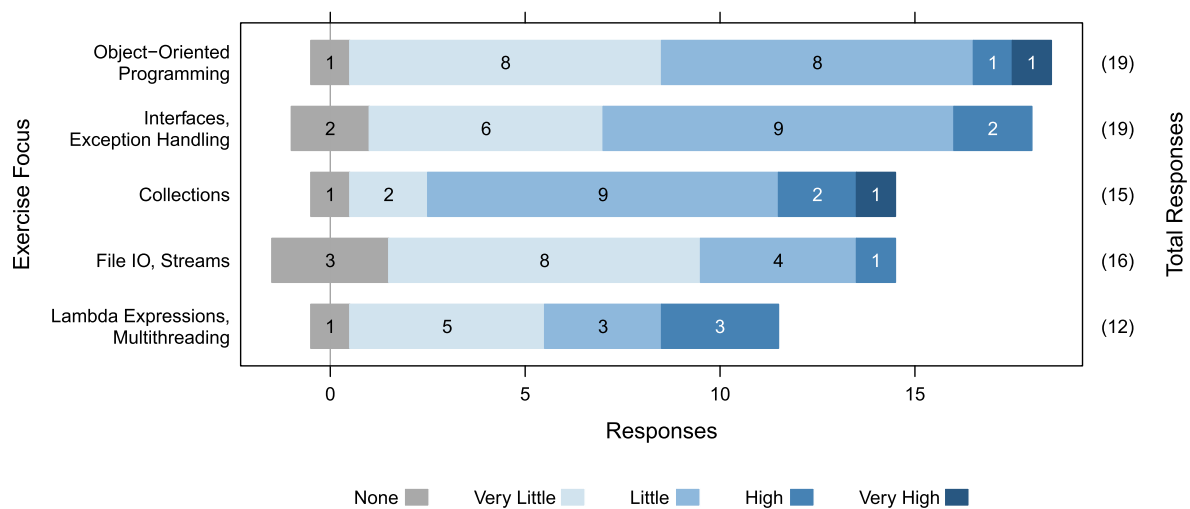


FIGURE 5. Students' ratings of the required effort to adapt ChatGPT's generated code to implementation tasks.

application scenario wasn't listed. Figure 6 summarizes all application scenarios ranked by accumulated share across all programming exercises. In the figure, the background color of cells indicate how often ChatGPT was used for an application scenario in a particular exercise, with darker colors representing higher usage.

According to the results, ChatGPT was primarily used by students to acquire background knowledge (68% of responses accumulated), to learn syntax and programming concepts (56% of responses accumulated), and to suggest suitable algorithms (47% of responses accumulated). On the other hand, it was least used for reviewing the own solution approach (28% responses accumulated). In the "Collections" exercise, students used ChatGPT much less for suggesting suitable algorithms (33.3% vs 55.25% average in preceding exercises) and for ideating solutions (20% vs

47.4% average in preceding exercises). In this exercise, however, students used ChatGPT more for improving (53.3% vs 31.6% average in preceding exercises) and reviewing (46.7% vs 23.7% average in preceding exercises) their own solutions. In no other exercise, ChatGPT was used for these two application areas that often.

In view of the most frequent usage of each application scenario across all programming exercises, the following picture emerges: The students most frequently used ChatGPT for (a) acquiring background knowledge and (b) learning syntax and programming concepts in the "Interfaces and Exception Handling" exercise, as well as (c) suggesting suitable algorithms in the "Lambda Expressions and Multithreading" exercise, for (d) solution ideation in the "File IO and Streams" exercise, for (e) creating the first solution approach in the "Object-Oriented Programming" and

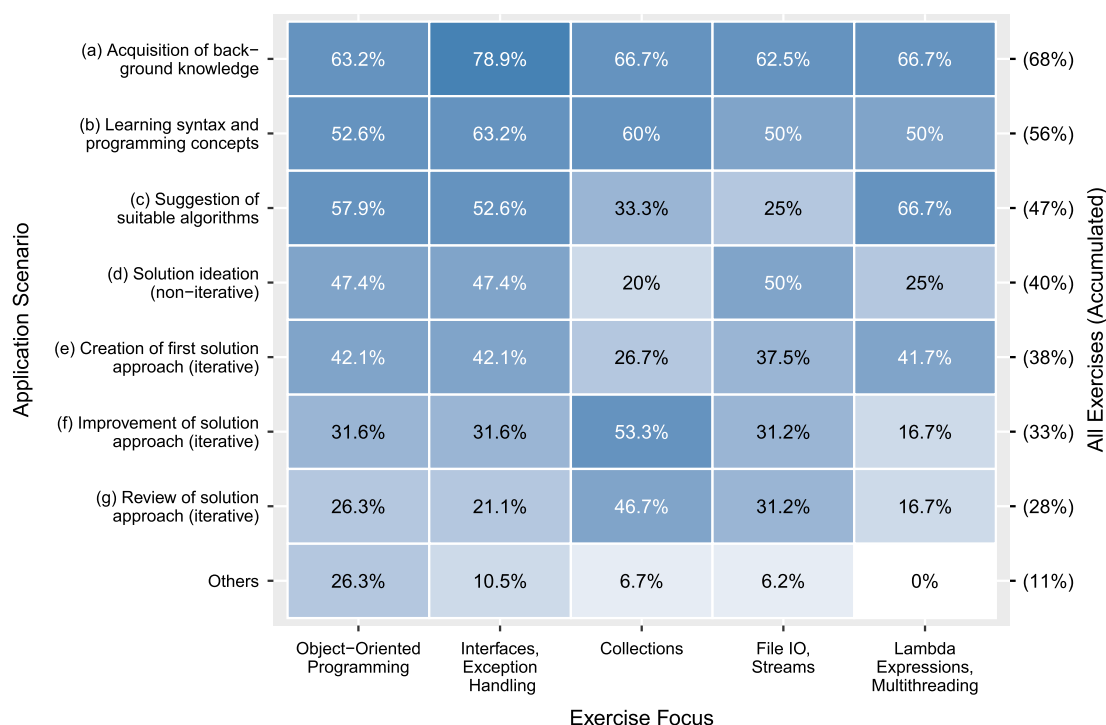


FIGURE 6. Students' application scenarios of ChatGPT in the programming exercise.

"Interfaces and Exception Handling" exercises, and for improving (f) and reviewing (g) the own solution approach in the "Collections" exercise.

1) OTHER APPLICATION AREAS

Students who indicated that they had used ChatGPT for other application area than those listed were asked to explain these scenarios in an open text field (Q12). For the "Interfaces and Exception Handling" and "Object-Oriented Programming" exercises, students mentioned that they used ChatGPT heavily for debugging and understanding misfunctions in their programs. Furthermore, they used ChatGPT throughout the whole programming course for interactive Q&A sessions to gain a deeper understanding of programming concepts and syntax.

E. REASONS FOR NOT USING CHATGPT (RQ5)

As shown in Table 2, three to six students per programming exercise choose to not use ChatGPT for the exercise. We identified five recurring themes in the students' answers, ordered here by the number of answers summarised therein.

1) FEAR OF NOT GARNERING PROGRAMMING PROFICIENCY

Six students expressed concerns that using ChatGPT for the exercises would prevent them in developing appropriate programming skills and that they eventually would not have enough knowledge to master the final exam. Furthermore, they argued that making their own mistakes and gaining experience when learning a new programming language helps

them better understand the nature of programming than receiving even a little support through ChatGPT. In particular, they mentioned that learning syntax by trial-and-error is easier for them than working with support.

2) PARTIALLY INCORRECT OR MISLEADING GENERATED CODE

Four students explained that ChatGPT's generated code solutions are often unsuitable. According to them, ChatGPT often provided strange ("no normal newbie would do it that way"), incorrect, and not well-suited answers. Several times, they had to modify the code significantly to get it to compile. One student reported that ChatGPT warned him about a missing parenthesis, which he tried to spot as a result, but it turned out that the problem was caused by something else, which ChatGPT hadn't yet been able to identify. Although students acknowledge that ChatGPT provides some assistance, they claim that its support frequently falls short, resulting in partially incorrect or misleading code suggestions.

3) DESIRE TO ESTABLISH AN INDEPENDENT PROGRAMMING STYLE

Three students expressed their desire to establish their own independent programming style. While they acknowledged that ChatGPT can guide them in the right direction, they often prefer to find more refined solutions for a particular implementation task through Google searches. Furthermore, these students noted that ChatGPT often provided only basic

information, and they wanted to dig deeper into the topic on their own.

4) FUNDAMENTAL REJECTION

Three students rejected ChatGPT outright, stating that they had never used it before and would not use it in the future. These cases were not further explained.

5) SKEPTICISM ABOUT BENEFITS

Two students claimed to see no benefit in using ChatGPT as they were certain to master the programming exercises better without it.

V. DISCUSSION

In light of our study's findings, we formulate three perspective directions for improving the suitability of LLM chatbots in programming education:

A. FOSTER PRACTICAL PROGRAMMING ON PROJECTS OF THE STUDENTS' OWN INTEREST

A programming course is commonly divided into several units, as is the case with the presented one. Individual concepts are practiced in isolation in each course unit, for example with practical tasks that students prepare independently. Practical tasks are typically not very complex, to not overwhelm students by new material. There is, however, no overarching application scenario where students can apply the teaching content directly, at best for something that they are genuinely interested in.

Chatbots like ChatGPT can play an important role in scaffolding and fading for programming education [23]. The term *scaffolding* refers to keeping the student's interest and focusing on the task while reducing the lecturer's burden. As a result, the task's difficulty does not exceed a student's cognitive ability. The term *fading* refers to a gradual reduction in such support until the student is able to manage the problem on his or her own. In cognitive load theory, *worked examples* [28] refer to instructional materials that provide step-by-step solutions to complex problems, helping learners better understand and reduce cognitive load when learning new concepts.

An application of these *worked examples* for programming education supported by LLM chatbots could look the following way: In order to motivate students and allow them to apply their knowledge gradually during the programming course, students define or select a practical example from their own interests that they wish to accomplish. The student's project need to be sufficiently complex, judged by the lecturer, so that the taught programming concepts inevitably need to be applied practically for the student's individual project. As opposed to focusing on individual language concepts and their syntax separately, students are trained using a set of *worked examples*. In the *scaffolding phase*, these examples are discussed with students. Hereby, the lecturer plays a key role by selecting appropriate *worked examples* and explaining them to the students. Subsequently, in the

fading phase, the students work on their selected projects and drive the development independently, thus inevitably applying the concepts presented in the previous phase. By providing immediate feedback based on the students' code, LLM chatbots can provide interactive support and solve coding problems not directly related to the students' concrete learning tasks. This prevents them from having to divide their attention between the learning task at hand and currently irrelevant topics [29] which may be addressed later in the course. In this phase, the lecturer gradually reduces his or her individual support, concentrating instead on guiding students through their implementation activities, resolving problems, and coming up with further worked examples to encourage self-directed learning.

B. GUIDE IN PROMPT ENGINEERING TO IMPROVE LLM RESPONSE QUALITY

In the realm of prompt engineering for LLM chatbots, Daun and Brings [24] accentuated the variability of ChatGPT's response quality based on prompt nuances, emphasizing the need for clarity, context, and precision. The *RTRI* structure presented by the University of Sydney [30] offers a blueprint for prompt engineering, encompassing *Role* for context-setting, *Task* for summarizing the task of the chatbot, *Requirements* for detailing expectations towards the response, and *Instructions* for defining how the chatbot should act on the prompt, e.g., create output, ask questions. For optimal LLM responses, students should be adept at prompt engineering principles, emphasizing specificity and iterative refinement based on feedback. An accompanying qualitative analysis of student prompts is essential to understanding their problem formulation strategies and gauging their adaptations.

C. ENHANCE DIGITAL LITERACY BY TRAINING HOW TO EVALUATE LLM RESPONSES FOR FACTFULNESS

Having identified reservations about using ChatGPT in research question RQ5, we emphasize the need for comprehensive training for students to critically evaluate LLM responses for factfulness. This training can equip them to distinguish between accurate, partially correct, and misleading answers, as well as to evaluate the quality of LLM chatbot's generated responses [20]. Additionally, it instills a deeper sense of digital literacy by bolstering their confidence in these tools. Students could benefit from practical workshops where they scrutinize LLM responses along with encouragement to verify information obtained from chatbots with reliable sources as a form of training. In addition, prompt engineering skills can further improve response accuracy. As a result of this approach to critical evaluation training, potential LLM shortcomings can be turned into valuable learning opportunities for students, preparing them to competently evaluate digital content.

VI. LIMITATIONS

We regard potential limitations to our study related to construct, internal, and external validity, some of which align

with concerns raised by researchers [31], [32] when assessing validity in qualitative research.

Limitations to the *internal validity* are notable, primarily due to different interpretations of qualitative statements among researchers, particularly when analyzing the students' open text answers (question Q12). This could have introduced inconsistencies in data interpretation. We tried to mitigate this threat by discussing the summarizations of the open text questions among the involved researchers. Additionally, there is the potential for students to interpret questions differently than intended, which could affect the accuracy of the data collected. To mitigate this threat, we asked the student representative to repeatedly ask the students if they had any questions regarding the survey.

External validity can be limited due to the relatively small sample size and the exclusive focus on the Java programming language. These factors might impede the generalizability of our findings to other programming languages. Moreover, variations in design of the exercises and implementation tasks within the Java programming course might influence students' perceptions of ChatGPT's suitability for other implementation tasks.

We see the foremost limitation to *construct validity* in the response bias introduced by students through providing either overly negative feedback or overestimating their own efforts to emphasize their contribution in the programming exercise, despite using ChatGPT. This could have influenced the accuracy of evaluating its suitability for implementation tasks, learning programming concepts, and its overall suitability (question Q7-Q9) as well as when rating the additional required efforts to adapt generated code to implementation tasks (question Q10). Second, students exhibited varying degrees of familiarity with ChatGPT, potentially influencing the quality of the prompts they formulated. Finally, in question Q6 of the survey, distinguishing between ChatGPT's sole effect on their learning progress and other contributing factors could have created ambiguity in the students' ratings.

VII. CONCLUSION

In this study, we examined students' experiences regarding the suitability of ChatGPT for different learning activities and its perceived impact on their learning progress in an undergraduate introductory Java programming course. During a five-week Java programming course, 18 to 22 part-time bachelor's degree students in information security participated. The exercises covered the fundamental concepts of the Java programming language. Each exercise was followed by an anonymous online survey containing Likert scale, closed, and open-ended questions.

With no negative ratings, the majority of students rated ChatGPT's effect on their learning progress as *positively* or *rather positively*. Except for the object-oriented programming exercise, its suitability for implementation tasks was rated predominantly as *rather suitable* or *suitable*. However, for each exercise there also were negative ratings. With only few exceptions, students generally rated ChatGPT as *rather*

suitable or *suitable* for learning programming concepts. This suggests that students had more confidence in ChatGPT's suitability to aid in learning programming concepts compared to its suitability for the implementation tasks of the exercises. The overall suitability of ChatGPT for the programming exercise also was predominantly rated as *positive*.

Students reported only little efforts required to adapt ChatGPT's generated code to the implementation tasks. Common application scenarios included acquiring background knowledge, learning syntax and programming concepts, suggesting suitable algorithms, and ideation of solutions. Concerns about programming proficiency, incomplete or misleading code, a desire for independent work, fundamental rejection, and an inability to recognize its benefits were reasons some students did not use ChatGPT.

Future work should focus on examining the experiences of a larger number of students, possibly also comparing ChatGPT's suitability for supporting students in programming courses other than Java. Additionally, it should evaluate the suitability of ChatGPT for each application scenario presented in this study individually and gather further empirical result from its use in different educational settings. A qualitative analysis of the students' prompts to ChatGPT could help analyze the structure and quality of their input. This way, educators could better teach students how to customize their input to ChatGPT to optimize their interactions.

REFERENCES

- [1] E. Adamopoulou and L. Moussiades, "Chatbots: History, technology, and applications," *Mach. Learn. Appl.*, vol. 2, Dec. 2020, Art. no. 100006.
- [2] J. F. Urquiza-Yllescas, S. Mendoza, J. Rodríguez, and L. M. Sánchez-Adame, "An approach to the classification of educational chatbots," *J. Intell. Fuzzy Syst.*, vol. 43, no. 4, pp. 5095–5107, Aug. 2022.
- [3] S. Hussain, O. A. Sianaki, and N. Ababneh, "A survey on conversational agents/chatbots classification and design techniques," in *Web, Artificial Intelligence and Network Applications, Advances in Intelligent Systems and Computing*, L. Barolli, M. Takizawa, F. Khafa, and T. Enokido, Eds. Cham, Switzerland: Springer, 2019, pp. 946–956.
- [4] M. Karyotaki, A. Drigas, and C. Skianis, "Chatbots as cognitive, educational, advisory & coaching systems," *Technium Social Sci. J.*, vol. 30, pp. 109–126, Apr. 2022.
- [5] C. W. Okonkwo and A. Ade-Ibijola, "Chatbots applications in education: A systematic review," *Comput. Educ., Artif. Intell.*, vol. 2, Oct. 2021, Art. no. 100033.
- [6] H. B. Essel, D. Vlachopoulos, A. Tachie-Menson, E. E. Johnson, and P. K. Baah, "The impact of a virtual teaching assistant (chatbot) on students' learning in Ghanaian higher education," *Int. J. Educ. Technol. Higher Educ.*, vol. 19, no. 1, p. 57, Nov. 2022.
- [7] M. C. Sáiz-Manzanares, R. Marticorena-Sánchez, L. J. Martín-Antón, I. González Díez, and L. Almeida, "Perceived satisfaction of university students with the use of chatbots as a tool for self-regulated learning," *Heliyon*, vol. 9, no. 1, Jan. 2023, Art. no. e12843.
- [8] H. Nguyen, "Role design considerations of conversational agents to facilitate discussion and systems thinking," *Comput. Educ.*, vol. 192, Jan. 2023, Art. no. 104661.
- [9] Y. Chen, S. Jensen, L. J. Albert, S. Gupta, and T. Lee, "Artificial intelligence (AI) student assistants in the classroom: Designing chatbots to support student success," *Inf. Syst. Frontiers*, vol. 25, no. 1, pp. 161–182, Feb. 2023.
- [10] L. Benotti, M. C. Martnez, and F. Schapachnik, "A tool for introducing computer science with automatic formative assessment," *IEEE Trans. Learn. Technol.*, vol. 11, no. 2, pp. 179–192, Apr. 2018.

- [11] I. G. Ndukwe, B. K. Daniel, and C. E. Amadi, "A machine learning grading system using chatbots," in *Artificial Intelligence in Education* (Lecture Notes in Computer Science), S. Isotani, E. Millán, A. Ogan, P. Hastings, B. McLaren, and R. Luckin, Eds. Cham, Switzerland: Springer, 2019, pp. 365–368.
- [12] T. T. Nguyen, A. D. Le, H. T. Hoang, and T. Nguyen, "NEU-chatbot: Chatbot for admission of National Economics University," *Comput. Educ., Artif. Intell.*, vol. 2, Oct. 2021, Art. no. 100036.
- [13] K. Lee, J. Jo, J. Kim, and Y. Kang, "Can chatbots help reduce the workload of administrative officers? Implementing and deploying FAQ chatbot service in a university," in *HCI International 2019—Posters* (Communications in Computer and Information Science), C. Stephanidis, Ed. Cham, Switzerland: Springer, 2019, pp. 348–354.
- [14] P. Smutny and P. Schreiberova, "Chatbots for learning: A review of educational chatbots for the Facebook Messenger," *Comput. Educ.*, vol. 151, 2020, Art. no. 103862. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0360131520300622>, doi: 10.1016/j.compedu.2020.103862.
- [15] S. Yang and C. Evans, "Opportunities and challenges in using AI chatbots in higher education," in *Proc. 3rd Int. Conf. Educ. E-Learning*. New York, NY, USA: Association for Computing Machinery, Nov. 2019.
- [16] I. Ozkaya, "Application of large language models to software engineering tasks: Opportunities, risks, and implications," *IEEE Softw.*, vol. 40, no. 3, pp. 4–8, May 2023.
- [17] I. Ozkaya, "The next frontier in software development: AI-augmented software development processes," *IEEE Softw.*, vol. 40, no. 4, pp. 4–9, Jul. 2023.
- [18] L. Jacques, "Teaching CS-101 at the dawn of ChatGPT," *ACM Inroads*, vol. 14, no. 2, pp. 40–46, Jun. 2023.
- [19] M. Welsh, "The end of programming," *Commun. ACM*, vol. 66, no. 1, pp. 34–35, 2022.
- [20] M. M. Rahman and Y. Watanobe, "ChatGPT for education and research: Opportunities, threats, and strategies," *Appl. Sci.*, vol. 13, no. 9, p. 5783, May 2023.
- [21] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, "Studying the effect of AI code generators on supporting novice learners in introductory programming," in *Proc. CHI Conf. Human Factors Comput. Syst.* New York, NY, USA: Association for Computing, Apr. 2023, pp. 1–23.
- [22] O. E. Chinonso, A. M.-E. Theresa, and T. C. Aduke, "ChatGPT for teaching, learning and research: Prospects and challenges," *Global Academic J. Humanities Social Sci.*, vol. 5, no. 2, pp. 33–40, Mar. 2023.
- [23] C. Bull and A. Kharrufa, "Generative artificial intelligence assistants in software development education: A vision for integrating generative artificial intelligence into educational practice, not instinctively defending against it," *IEEE Softw.*, vol. 41, no. 2, pp. 52–59, Apr. 2024.
- [24] M. Daun and J. Brings, "How ChatGPT will change software engineering education," in *Proc. Conf. Innov. Technol. Comput. Sci. Educ. V. I.* New York, NY, USA: Association for Computing Machinery, Jun. 2023, pp. 110–116.
- [25] E. L. Ouh, B. K. S. Gan, K. J. Shim, and S. Wlodkowski, "ChatGPT, can you generate solutions for my coding exercises? An evaluation on its effectiveness in an undergraduate Java programming course," in *Proc. Conf. Innov. Technol. Comput. Sci. Educ. V. I.* New York, NY, USA: Association for Computing Machinery, Jun. 2023, pp. 54–60.
- [26] J. Savelka, A. Agarwal, C. Bogart, Y. Song, and M. Sakr, "Can generative pre-trained transformers (GPT) pass assessments in higher education programming courses?" in *Proc. Conf. Innov. Technol. Comput. Sci. Educ. V. I.* New York, NY, USA: Association for Computing Machinery, Jun. 2023, pp. 117–123.
- [27] S. Adolph, W. Hall, and P. Kruchten, "Using grounded theory to study the experience of software development," *Empirical Softw. Eng.*, vol. 16, no. 4, pp. 487–513, Aug. 2011.
- [28] J. Sweller, P. Ayres, and S. Kalyuga, "The worked example and problem completion effects," in *Cognitive Load Theory* (Explorations in the Learning Sciences, Instructional Systems and Performance Technologies), J. Sweller, P. Ayres, and S. Kalyuga, Eds. New York, NY, USA: Springer, 2011, pp. 99–109.
- [29] J. Sweller, P. Ayres, and S. Kalyuga, "The split-attention effect," in *Cognitive Load Theory* (Explorations in the Learning Sciences, Instructional Systems and Performance Technologies). New York, NY, USA: Springer, 2011, pp. 111–128.
- [30] *Prompt Engineering for Educators—Making Generative AI Work for You*, 2023, Univ. Sydney, Sydney, NSW, Australia, Oct. 2024.
- [31] N. Golafshani, "Understanding reliability and validity in qualitative research," *Qualitative Rep.*, vol. 8, no. 4, pp. 597–606, Jan. 2015.
- [32] A. J. Onwuegbuzie and N. L. Leech, "Validity and qualitative research: An oxymoron?" *Qual. Quantity*, vol. 41, no. 2, pp. 233–249, Feb. 2007.



PHILIPP HAINDL received the Ph.D. degree in computer science from Johannes Kepler University Linz, in 2021. He is currently a Lecturer in software engineering with St. Pölten University of Applied Sciences. He has more than 15 years of practical experience in industrial software projects as technology consultant, software engineer, and architect. His current research interests include empirical software engineering, software security, safety, and software quality operationalization.

He regularly functions as a reviewer for international software engineering conferences and journals.



GERALD WEINBERGER received the Dipl.-Ing. (M.Sc. equivalent) and B.Sc. degrees in IT security from St. Pölten University of Applied Sciences. He is currently a Lecturer with the Department of Informatics and Security, St. Pölten University of Applied Sciences.

...